

Manual técnico — Galsoft Helpdesk

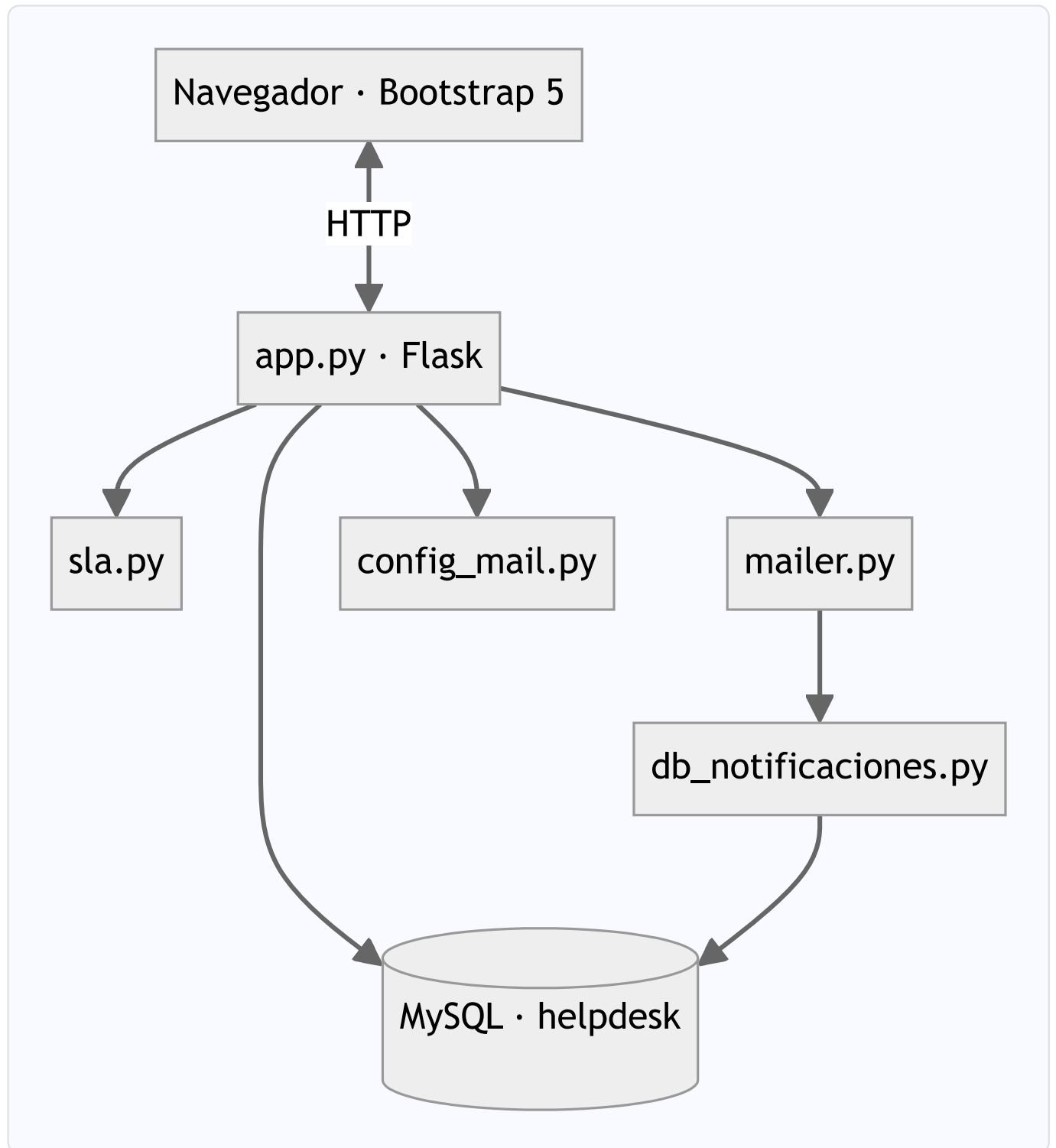
1. Objetivo del sistema

Aplicación web de mesa de ayuda (helpdesk) que permite:

- A los **clientes**: abrir peticiones, consultar estado y comentar.
- A los **agentes**: gestionar tickets, asignarse incidencias, responder y controlar SLA.
- A los **administradores**: gestionar usuarios, categorías y revisar el historial de correos.

La base de datos se entrega completa en `helpdesk.sql` (export de phpMyAdmin).

2. Arquitectura general



Flujo resumido: el navegador habla con Flask (`app.py`); la lógica auxiliar está en `sla.py` , `mailer.py` y `config_mail.py` ; los correos se registran con `db_notificaciones.py` ; todo persiste en MySQL.

- **Presentación:** plantillas Jinja2 en `templates/` + estilos en `static/css/styles.css` .
- **Lógica de negocio:** concentrada en `app.py` , con módulos auxiliares para SLA y correo.
- **Persistencia:** MySQL mediante `mysql.connector` , cursor global `micursor` .

3. Estructura de archivos

Ruta	Descripción
<code>app.py</code>	Aplicación Flask: rutas, sesión, consultas SQL, orquestación
<code>sla.py</code>	Cálculo de plazos SLA y evaluación (en plazo / riesgo / vencido)
<code>mailer.py</code>	Envío SMTP y avisos automáticos (ticket nuevo, comentarios, etc.)
<code>config_mail.py</code>	Carga <code>mail.env</code> y variables de entorno de correo
<code>db_notificaciones.py</code>	Inserción y listado de <code>notificaciones</code>
<code>mail.env</code>	Configuración SMTP real (no subir a repositorio; crear desde <code>mail.env.example</code>)
<code>mail.env.example</code>	Plantilla de configuración de correo
<code>helpdesk.sql</code>	Base de datos completa (estructura + datos) para importar en MySQL
<code>templates/</code>	Vistas HTML (base, index, detalle, dashboard, login, etc.)
<code>static/css/styles.css</code>	Estilos Galsoft (paleta pastel, sidebar, tarjetas)
<code>static/LogoGalsoft.png</code>	Logo corporativo
<code>MANUAL_USUARIO.md</code>	Manual de uso para usuarios finales
<code>MANUAL_TECNICO.md</code>	Este documento

4. Base de datos `helpdesk`

4.1 Instalación de la base de datos

1. Abrir **phpMyAdmin** (o consola MySQL).
2. Pestaña **Importar** → seleccionar `helpdesk.sql` .
3. Ejecutar. El script crea la base `helpdesk` , las tablas y los datos de ejemplo.

4.2 Tablas del sistema

`usuarios`

Columna	Tipo	Descripción
<code>id</code>	INT PK AI	Identificador
<code>nombre</code>	VARCHAR	Nombre completo
<code>email</code>	VARCHAR	Correo (login único)
<code>password</code>	VARCHAR	Hash bcrypt (<code>werkzeug.security</code>)
<code>rol</code>	VARCHAR	<code>cliente</code> , <code>agente</code> o <code>admin</code>
<code>reset_token</code>	VARCHAR(64) NULL	Token para enlace de recuperación de contraseña
<code>reset_token_expira</code>	DATETIME NULL	Caducidad del token (24 h)

tickets

Columna	Tipo	Descripción
<code>id</code>	INT PK AI	Número de ticket
<code>titulo</code>	VARCHAR	Asunto breve
<code>descripcion</code>	TEXT	Detalle de la incidencia
<code>estado</code>	VARCHAR	<code>Abierto</code> , <code>En proceso</code> , <code>Cerrado</code> (por defecto Abierto)
<code>prioridad</code>	VARCHAR	<code>baja</code> , <code>media</code> , <code>alta</code>
<code>archivo</code>	VARCHAR	Nombre del fichero adjunto
<code>archivo_blob</code>	LOB	Contenido binario del adjunto
<code>usuario_id</code>	INT FK	Cliente que abrió el ticket
<code>agente_id</code>	INT FK	Agente asignado; NULL = sin asignar
<code>categoria_id</code>	INT FK	Categoría del ticket
<code>cerrado_en</code>	DATETIME	Fecha/hora de cierre
<code>sla_respuesta_limite</code>	DATETIME	Plazo máximo de primera respuesta
<code>sla_resolucion_limite</code>	DATETIME	Plazo máximo de resolución
<code>primera_respuesta_en</code>	DATETIME	Primera respuesta pública de agente/admin

Columna	Tipo	Descripción
<code>creado_en</code>	DATETIME	Fecha de alta

categorias

Columna	Tipo	Descripción
<code>id</code>	INT PK AI	Identificador
<code>nombre</code>	VARCHAR	Nombre único (General, Hardware, Software, etc.)

ticket_historial

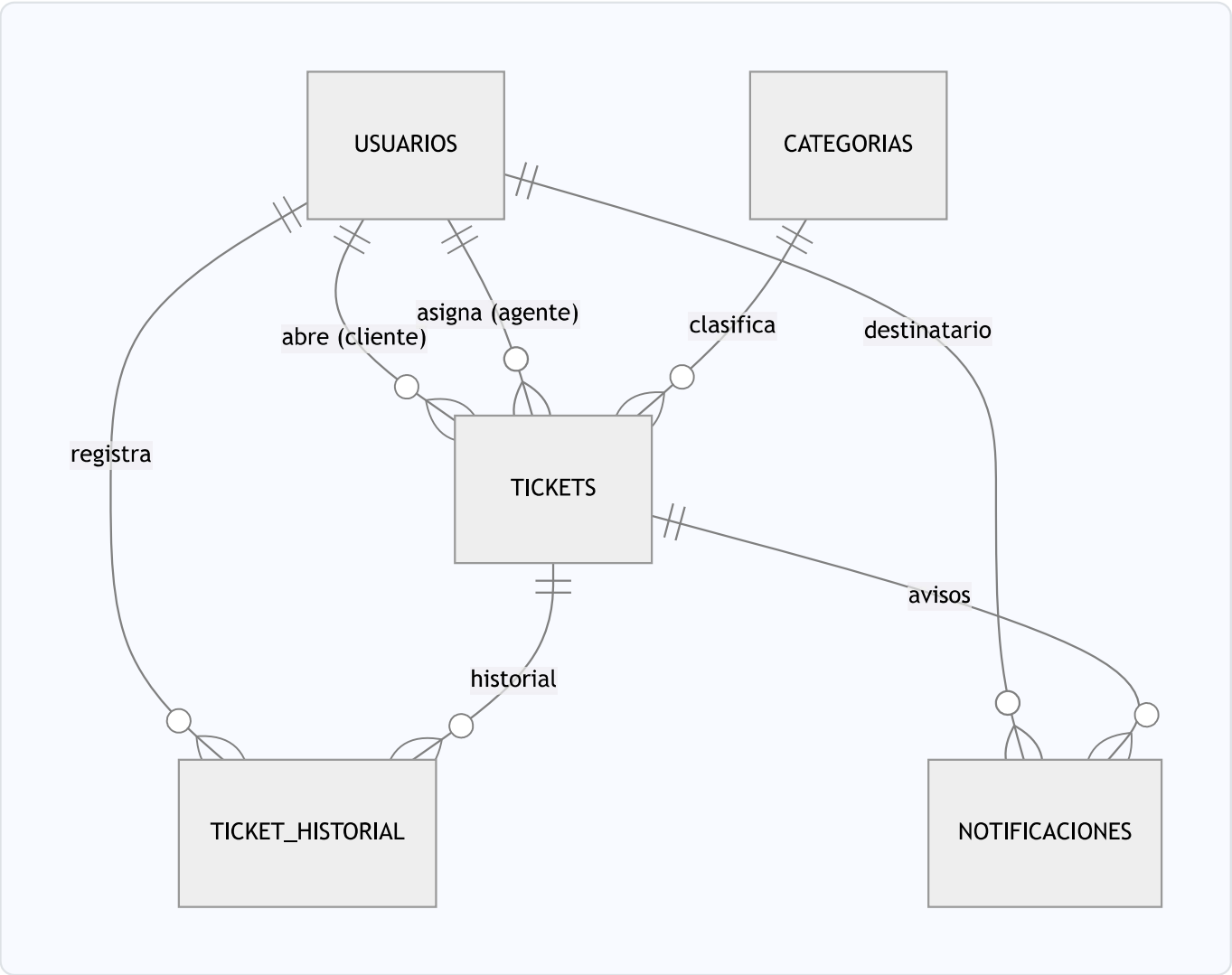
Columna	Tipo	Descripción
<code>id</code>	INT PK AI	Identificador
<code>ticket_id</code>	INT FK	Ticket relacionado
<code>usuario_id</code>	INT FK	Quién realizó la acción
<code>tipo</code>	VARCHAR	<code>creado</code> , <code>comentario</code> , <code>estado</code> , <code>prioridad</code> , <code>asignacion</code> , <code>archivo</code> , etc.
<code>detalle</code>	TEXT	Texto descriptivo
<code>es_interno</code>	TINYINT	1 = nota solo visible para agente/admin
<code>fecha</code>	DATETIME	Marca temporal

notificaciones

Columna	Tipo	Descripción
<code>id</code>	INT PK AI	Identificador
<code>ticket_id</code>	INT FK	Ticket relacionado
<code>email</code>	VARCHAR	Destinatario
<code>tipo</code>	VARCHAR	Evento (<code>ticket_creado_cliente</code> , <code>comentario_agente</code> , etc.)
<code>asunto</code>	VARCHAR	Asunto del correo
<code>cuerpo</code>	TEXT	Cuerpo del mensaje

Columna	Tipo	Descripción
enviado	TINYINT	1 = enviado correctamente
error_msg	VARCHAR	Error SMTP si falló
creado_en	DATETIME	Fecha del intento de envío

4.3 Diagrama entidad-relación (simplificado)



5. Rutas Flask (app.py)

Cada ruta lleva un comentario `# RUTA:` en el código fuente.

Ruta	Función	Acceso
/	Listado de tickets con filtros	Sesión requerida
/ticket/nuevo	Crear ticket	Cliente+
/login	Autenticación	Público

Ruta	Función	Acceso
<code>/register</code>	Alta de cliente	Público
<code>/logout</code>	Cerrar sesión	Sesión
<code>/descargar/<id></code>	Descarga adjunto BLOB	Según permisos
<code>/ticket/<id></code>	Detalle, comentarios, gestión	Según rol
<code>/usuarios</code>	Listado usuarios	Admin
<code>/usuario/editar/<id></code>	Editar usuario	Admin
<code>/usuario/eliminar/<id></code>	Eliminar usuario (POST con confirmación UI)	Admin
<code>/categorias</code>	CRUD categorías	Admin
<code>/dashboard</code>	Estadísticas y SLA	Agente, admin
<code>/notificaciones</code>	Historial de correos	Admin
<code>/manual/usuario</code>	Manual de usuario (HTML)	Público
<code>/manual/tecnico</code>	Manual técnico (HTML)	Público
<code>/olvide</code>	Solicitar enlace de recuperación por correo	Público
<code>/cuenta/cambiar-contrasena</code>	Enviar enlace de cambio (usuario logueado)	Autenticado
<code>/reset/<token></code>	Formulario nueva contraseña (token del correo)	Público

Sesión Flask (`session`): `usuario_id` , `usuario_nombre` , `usuario_rol` .

Clave secreta: `app.secret_key = os.urandom(24)` — las sesiones se invalidan al reiniciar el servidor (comportamiento documentado en FAQ).

6. Módulos auxiliares

6.1 `sla.py`

- Constante `SLA_HORAS` : plazos por prioridad (baja/media/alta).
- `calcular_limites_sla(prioridad, creado_en)` → tupla (límite respuesta, límite resolución).
- `evaluar_sla(ticket)` → diccionario con estados, límites (`limite_respuesta` , `limite_resolucion`), `estado_global` y `deadline_activo` .
- `RIESGO_FRACCION = 0.25` : último 25 % del plazo = «en riesgo» (badge SLA).
- `marcar_primera_respuesta()` al primer comentario público de agente/admin.

- `etiqueta_estado(codigo)` → texto legible para plantillas (En plazo, Vencido, etc.).

6.2 `mailer.py` + `config_mail.py`

- Lee `mail.env` (IONOS, Gmail u otro SMTP).
- `MAIL_ENABLED=0` desactiva envío; la app sigue funcionando.
- Eventos que disparan correo: ticket nuevo, comentarios, cambio estado, asignación.
- Cada intento se registra en `notificaciones` vía `db_notificaciones.py`.

6.3 `db_notificaciones.py`

- Detecta columnas reales de `notificaciones` (`creado_en` , `fecha_envio` , `mensaje` , etc.).
- Normaliza filas para plantillas aunque la tabla tenga columnas de esquemas anteriores mezcladas.

7. Seguridad y buenas prácticas implementadas

Aspecto	Implementación
Contraseñas de acceso	<code>generate_password_hash</code> / <code>check_password_hash</code> (Werkzeug) en <code>password</code>
Recuperación de contraseña	Token <code>secrets.token_urlsafe(32)</code> + caducidad 24 h; enlace en correo vía <code>mailer.enviar_recuperacion_contrasena</code>
SQL injection	Consultas parametrizadas con <code>%s</code> y tuplas de valores
Archivos	<code>secure_filename</code> en subida; almacenamiento en BLOB
Autorización	Comprobación de <code>session</code> y <code>usuario_rol</code> en cada ruta sensible
Notas internas	Filtro <code>es_interno = 0</code> en historial para clientes
Tickets cliente	<code>WHERE t.usuario_id = %s</code> en consultas de detalle/listado

8. Interfaz de usuario

- **Bootstrap 5.3.3** (CDN).
- **Sidebar** fijo bajo navbar (`top: 64px`); toggle móvil con ☰.
- **Listado en tarjetas** (`index.html`): badge SLA arriba a la derecha; estado del ticket en el cuerpo de la tarjeta; contadores SLA en vivo.
- **Filtro Jinja** `fecha_es` : formato `DD-MM-AAAA HH:MM` .
- **Paleta**: verde (ok/abierto/baja/cumplido), amarillo (proceso/media/riesgo), naranja (contador < 2 h), rojo (cerrado/alta/vencido).

8.1 Contadores SLA en listado de tickets

Solo visibles para **agente** y **admin** en `templates/index.html`.

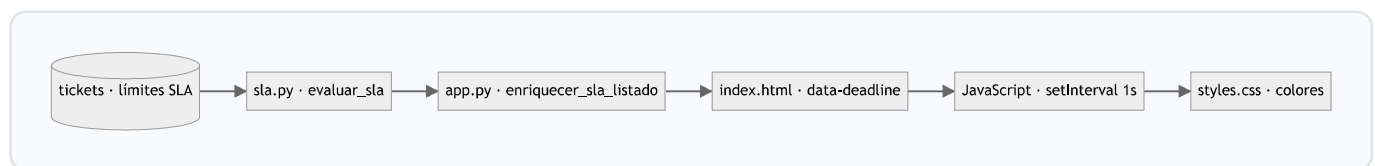
Estructura de cada tarjeta (zona SLA):

1. Badge **SLA:** + `estado_global` (calculado en servidor con `evaluar_sla`).
2. Línea **1ª respuesta:** — cuenta atrás hasta `sla_respuesta_limite` o «Cumplida» si existe `primera_respuesta_en`.
3. Línea **Resolución:** — «Cerrada en plazo» (verde) o «Cerrada fuera de plazo» (rojo) según `estado_resolucion`; cuenta atrás si sigue abierto.

Lógica en cliente (JavaScript embebido en `index.html`):

- Cada línea activa lleva `data-deadline` con fecha límite ISO.
- `setInterval(..., 1000)` recalcula el texto cada segundo.
- Umbral urgente: $2 * 60 * 60 * 1000$ ms (2 horas).
- Clases CSS en `static/css/styles.css`:
 - `.sla-timer` — texto negro por defecto.
 - `.sla-timer-urgente` — naranja (`#e67e22`) si quedan < 2 h.
 - `.sla-timer-vencido` — rojo si el plazo ya pasó.
 - `.sla-timer-cumplido` — verde si cumplida/cerrada.

Flujo de datos:



Plantillas principales: `base.html`, `index.html`, `ticket_detalle.html`, `dashboard.html`, `login.html`, `register.html`, `olvide_contrasena.html`, `reset_password.html`, `solicitar_cambio_contrasena.html`, `new_ticket.html`, `usuarios.html`, `editar_usuario.html`, `categorias.html`, `notificaciones.html`, `manual_view.html`.

9. Instalación y ejecución

9.1 Obtener el proyecto

1. Clonar el repositorio desde GitHub (rama `main`). GitHub Desktop o `git clone` crearán la carpeta en la ruta que elija cada usuario.
2. Abrir esa carpeta: es la raíz del proyecto (contiene `app.py`, `helpdesk.sql` y `templates/`).

9.2 Requisitos

- Python 3.x
- MySQL / MariaDB (servicio activo)
- Paquetes: `flask`, `mysql-connector-python`, `werkzeug`

```
pip install flask mysql-connector-python werkzeug
```

9.3 Base de datos

1. Importar `helpdesk.sql` en phpMyAdmin (crea la base, tablas y datos).
2. Verificar conexión en `app.py` :

```
mibd = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="",  
    database="helpdesk"  
)
```

9.4 Correo (opcional)

Copiar la plantilla y editarla (Windows: `copy` ; Linux/Mac: `cp`):

```
copy mail.env.example mail.env
```

Editar `mail.env` con datos SMTP (ejemplo IONOS: `smtp.ionos.es` , puerto 587, STARTTLS).

9.5 Arranque

Desde la **raíz del repositorio clonado** (carpeta con `app.py`):

```
python app.py
```

Abrir <http://127.0.0.1:5000>

10. Flujos de negocio relevantes

10.1 Creación de ticket

1. POST `/ticket/nuevo` → INSERT en `tickets` con límites SLA.
2. `registrar_historial(..., 'creado', ...)` .
3. `mailer.on_ticket_nuevo()` → cliente y equipo.

10.2 Tomar ticket (agente)

1. UPDATE `agente_id` , posible cambio a «En proceso».
2. Historial tipo `asignacion` .
3. Notificación al agente si aplica.

10.3 Comentario agente (público)

1. INSERT historial `comentario` .
2. `marcar_primera_respuesta` si es la primera.
3. `mailer` avisa al cliente.

10.4 Cierre

1. Estado `Cerrado` , `cerrado_en = NOW()` .
2. Historial + correo al cliente.

10.5 Registro de usuario (cliente)

1. POST `/register` → `generate_password_hash(password)` .
2. INSERT en `usuarios` (nombre, email, password, rol= `cliente`).

10.6 Recuperación y cambio de contraseña por correo

Requiere `MAIL_ENABLED=1` y `mail.env` configurado (IONOS, etc.):

1. **POST** `/olvide` — Usuario introduce email. Si existe, genera token, guarda `reset_token` + `reset_token_expira` y envía correo con enlace `{APP_BASE_URL}/reset/{token}` .
2. **POST** `/cuenta/cambiar-contrasena` — Usuario logueado; mismo mecanismo al email de su ficha.
3. **GET/POST** `/reset/<token>` — Valida token no caducado; POST actualiza `password` y borra token.
4. Mensaje genérico en `/olvide` (no revela si el email existe).

Administrador: POST `/usuario/editar/<id>` con `password` no vacío sigue permitiendo asignar contraseña manualmente.

10.7 Gestión y eliminación de usuarios (admin)

1. **GET** `/usuarios` muestra listado con búsqueda (`q`) por nombre/email.
2. **POST** `/usuario/editar/<id>` actualiza nombre, email, rol y contraseña opcional.
3. **POST** `/usuario/eliminar/<id>` elimina la cuenta seleccionada tras confirmación en interfaz.
4. Protección de seguridad: el admin logueado no puede eliminar su propio usuario en esa sesión.

10.8 Contadores SLA en listado (agente/admin)

1. `listar_tickets_consulta()` obtiene tickets con JOINS.
2. `enriquecer_sla_listado()` añade `t['sla'] = evaluar_sla(t)` por ticket.
3. `index.html` renderiza:
 - Badge `estado_global` arriba a la derecha.
 - `data-deadline` con `limite_respuesta` y `limite_resolucion` .
4. JavaScript en el navegador actualiza cada segundo el texto «Quedan X» / «Vencido hace X».
5. Colores: negro (> 2 h), naranja (< 2 h), rojo (vencido), verde (cumplida/cerrada).

11. Consultas SQL destacadas

- **Listado base:** `SQL_TICKET_BASE` — JOIN cliente, agente y categoría.

- **Filtro SLA vencido:** compara `NOW()` con `sla_respuesta_limite` / `sla_resolucion_limite` .
 - **Dashboard:** agregaciones `COUNT` , `GROUP BY` por agente y categoría.
 - **Historial:** orden `fecha DESC` , filtro `es_interno` para clientes.
-

12. Limitaciones conocidas

- Conexión MySQL única global (no pool); adecuado para demo/local.
 - `secret_key` aleatoria reinicia sesiones al reiniciar Flask.
 - Adjuntos en BLOB (no escalable para ficheros muy grandes).
 - Sin API REST ni websockets; interfaz 100 % servidor-renderizada.
 - Correo depende de credenciales SMTP en `mail.env` ; la recuperación de contraseña **requiere** `MAIL_ENABLED=1` .
 - Tokens de reset caducan a las 24 horas y se invalidan al usarse.
-

Ver en la aplicación: <http://127.0.0.1:5000/manual/tecnico> (también desde el pie de página).
